



Projektportfolio

IU Internationale Hochschule

Studiengang: M.Sc. Informatik

Projektdokumentation

Projekt Software Engineering

DLMCSPSE01_D_CF

GitHub: <https://github.com/micael92/FlowWorkbench>

Download: <https://github.com/micael92/FlowWorkbench/releases/tag/v0.1.0>

Michael Hilfer

Matrikelnummer: 3205669

Tutor: Prof. Dr. Dirk Simon

Inhaltsverzeichnis

Abbildungsverzeichnis	III
1. Einleitung.....	1
1.1 Motivation.....	1
1.2 Ziel der Arbeit	2
1.3 Methodik.....	2
1.4 Dokumentation	3
2. Projekt	4
2.1 Übersicht.....	4
2.2 Risikomanagement.....	4
2.3 Zeitplanung.....	6
Literaturverzeichnis.....	7

Abbildungsverzeichnis

Abb. 1: Kontextdiagramm FlowWorkbench	4
Abb. 2: Ergebnisse der Risikoanalyse	5
Abb. 3: Zeitplanung des Projekts	6

1. Einleitung

1.1 Motivation

Die Erkennung von Angriffen und Eindringlingen in einem nicht allgemein zugänglichen Netzwerk ist eine effektive Maßnahme zum Schutz der im Netzwerk erreichbaren Systeme. Zur Erkennung von sicherheitsrelevanten Aktivitäten in einem Netzwerk werden deswegen sogenannte Intrusion Detection Systeme (IDS) eingesetzt, die durch die Analyse von Netzwerkdaten Angriffe erkennen können. Bei der Erkennung kann zwischen signaturbasierten und anomaliebasierten IDS unterschieden werden. Während signaturbasierte Systeme bekannte Angriffsmuster zur Erkennung nutzen, können in anomaliebasierten Systemen unter anderem Ansätze aus dem maschinellen Lernen genutzt werden, um Anomalien zu erkennen. Hierbei werden Modelle auf Basis vorhandener Netzwerkdaten trainiert, um dann zur Erkennung bzw. Klassifikation von Netzwerkdaten eingesetzt zu werden (Schrötter et al., 2024, S. 1–2).

Zur Entwicklung und Evaluation von Modellen des überwachten maschinellen Lernens im IDS-Kontext werden im wissenschaftlichen Umfeld regelmäßig öffentlich zur Verfügung gestellte Datensätze genutzt (Ring et al., 2019, S. 147). Diese werden von verschiedenen Institutionen erzeugt und veröffentlicht. So ist beispielsweise der 2017 vom Canadian Institute for Cybersecurity veröffentlichte Datensatz CIC-IDS-2017 ein im IDS-Kontext sehr verbreiteter Datensatz (Canadian Institute for Cybersecurity, 2018; Schrötter et al., 2024, S. 5–6).

Bei der Nutzung derartiger Datensätze spielen verschiedene Aspekte eine Rolle. Grundlegend besteht schon bei Verfahren wie z.B. der Auswahl eines geeigneten Datensatzes, dem Vergleich von Datensätzen oder auch der Kombination von Datensätzen der Bedarf, gewählte Datensätze einzusehen und grundlegende strukturelle oder auch statistische Eigenschaften zu ermitteln. Für eine weitere Nutzung im Bereich des maschinellen Lernens werden für einen Datensatz auch weitere Schritte im Rahmen der sogenannten Datenvorverarbeitung (engl. preprocessing) notwendig, in denen die Daten für Training und Validierung eines ML-Modells vorbereitet werden (Manocchio et al., 2025, S. 3).

In den genannten Anwendungsfällen im Bereich der ersten Betrachtung von Datensätzen können gängige Standardprogramme wie Tabellenkalkulationsprogramme und Texteditoren an ihre Leistungsgrenzen stoßen oder unkomfortabel und unpraktikabel werden. Auch der zweitgenannte Anwendungsfall der Vorverarbeitung ist mit derartigen Anwendungen schwer umzusetzen.

Für den Anwendungsfall der ersten Betrachtung und Einschätzung eines Datensatzes könnte eine leichtgewichtige Anwendung hilfreich sein, die es ermöglicht, Datensätze schnell und ohne Programmieraufwand einer ersten Betrachtung zu unterziehen und dabei erste grundlegende Informationen zu gewinnen. Weiterführend könnte eine solche Anwendung ggf. auch erste Schritte eines folgenden Preprocessings übernehmen.

1.2 Ziel der Arbeit

Ziel dieser Arbeit ist es, eine funktionale Version einer Anwendung zu entwickeln, die den eingangs genannten Anwendungsfall der „ersten Betrachtung und oberflächlichen Analyse eines Datensatzes“ ermöglichen kann. Ebenfalls soll die Entwicklung zeigen, inwiefern ausgewählte Schritte aus dem Preprocessing in eine solche Anwendung integriert werden könnten. Mit dem Entwicklungsprojekt sollen somit die folgenden der Entwicklung zugrunde liegenden Projektfragen beantwortet werden:

1. Inwieweit kann eine grafische Desktopanwendung die grundlegende Betrachtung und erste Analyse von IDS-Datensätzen ohne individuellen Programmieraufwand ermöglichen?
2. Inwieweit lassen sich in eine solche Anwendung Schritte aus der Vorverarbeitung eines IDS-Datensatzes integrieren?

1.3 Methodik

Die zuvor gestellten Projektfragen sollen in einem Software-Engineering-Projekt beantwortet werden. Hierbei soll ein Software-Artefakt in Form einer Software-Anwendung mit grafischer Benutzeroberfläche konzipiert, entwickelt und qualitätsgesichert werden. Grundlegend wird das Projekt dazu in drei Phasen durchgeführt.

In der ersten Phase wird das Projekt initialisiert und konzipiert. Hier werden die Rahmenbedingungen für das Projekt definiert. Dabei wird eine Zeitplanung aufgestellt, eine geeignete Form des Risikomanagements wird eingeführt und Stakeholder werden ermittelt und analysiert. Insbesondere wird hier jedoch das Requirements Engineering durchgeführt, um die Anforderungen für die zu entwickelnde Anwendung zu ermitteln. Anforderungen werden in Form von User Stories genauer deklariert und mit der MoSCoW-Methode priorisiert. Auf Basis der Anforderungen wird die Anwendung dann genauer spezifiziert. Hierbei werden Geschäftsobjekte, -prozesse, -regeln und Systemschnittstellen grundlegend definiert. Zur Darstellung der Ergebnisse dieser Phase werden verschiedene Diagrammtypen eingesetzt. In dieser Phase werden somit bewährte Artefakte aus dem Projektmanagement und dem Requirements Engineering erarbeitet.

Nach der Konzeptionsphase folgt die Erarbeitungsphase. In dieser Phase wird zunächst eine geeignete Softwarearchitektur erarbeitet. Dabei werden grundlegende Technologien in Form von Programmiersprachen, Frameworks und Bibliotheken ausgewählt, bevor die Software entwickelt wird. Eine Ergänzung von Frameworks und Bibliotheken wird jedoch auch während der konkreten Anwendungsentwicklung ermöglicht.

Die Entwicklung der Anwendung wird in einem iterativen und inkrementellen Vorgehen umgesetzt. Das Vorgehen wurde in diesem Projekt aufgrund der hohen Flexibilität und der Möglichkeit einer schrittweisen Entwicklung gewählt. Grundsätzlich wird zunächst die Entwicklung eines funktionalen

Kerns für die Anwendung angestrebt, welcher dann iterativ um zusätzliche funktionale Inkremente ergänzt wird (Broy & Kuhrmann, 2021, S. 89–90). Frühe Inkremente enthalten demnach typischerweise die wichtigsten Funktionen einer Anwendung. Durch eine schrittweise Umsetzung einzelner Inkremente können bereits während der Entwicklung lauffähige Zwischenstände erzeugt und frühzeitig getestet werden. Erkenntnisse aus einer Iteration können in den nächsten Entwicklungsschritten berücksichtigt werden und bei Bedarf auch zu Anpassungen in der Architektur führen (Sommerville, 2012, S. 59–60).

Die zu entwickelnden Inkremente sollen in diesem Projekt an den funktionalen Anforderungen ausgerichtet werden. In einem Entwicklungsschritt wird eine funktionale Anforderung zur Umsetzung ausgewählt. Anschließend werden zur Umsetzung notwendige Architekturentscheidungen getroffen und der Code wird implementiert. Nach der Implementierung werden bereits innerhalb der Iteration Tests zur Überprüfung und Qualitätssicherung durchgeführt. Dabei werden sowohl automatisierte Tests als auch manuelle Tests durchgeführt, sodass nach jeder Iteration ein funktionales Inkrement entsteht.

Nach Abschluss der Entwicklung wird die entwickelte Anwendung in einer letzten Phase abschließend qualitätsgesichert. Dazu wird die Teststrategie finalisiert und entsprechende Tests werden durchgeführt und dokumentiert. Bei Bedarf werden letzte Anpassungen zur Erfüllung der Anforderungen durchgeführt. Zuletzt wird die Zielerreichung des Projektes bewertet und die eingangs formulierten Projektfragen werden beantwortet. Dabei soll das Projekt kritisch reflektiert werden und Limitierungen sollen dargestellt werden. Die Phase wird mit einem Ausblick zur möglichen Weiterentwicklung der Anwendung abgeschlossen.

1.4 Dokumentation

Die Arbeit besteht aus den folgenden zusammenhängenden bzw. aufeinander aufbauenden Dokumenten:

- Projektdokument
- Anforderungsdokument
- Spezifikationsdokument
- Architekturdokument
- Testdokument

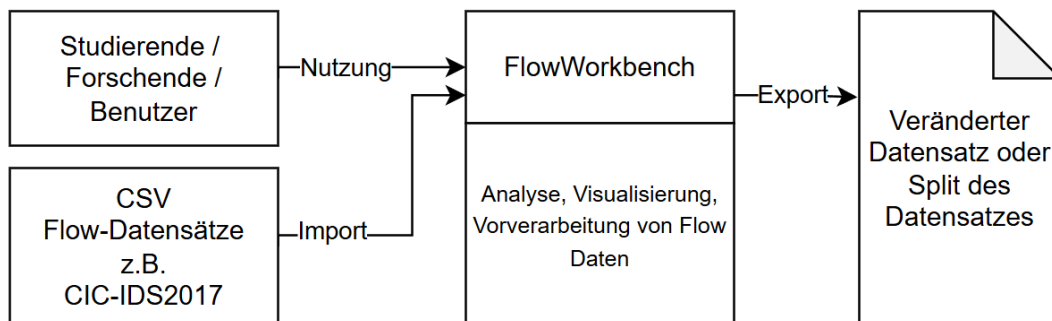
2. Projekt

2.1 Übersicht

Im Projekt wird eine im Folgenden als FlowWorkbench bezeichnete Anwendung entwickelt, die Benutzern erste Analysen, Visualisierungsmöglichkeiten und grundlegende Schritte zur Vorverarbeitung von Datensätzen aus dem Bereich der Intrusion Detection Forschung ermöglicht und damit eine weitere Nutzung in Machine-Learning-Prozessen erleichtert. Der Kundennutzen (Customer Value) liegt damit in der Möglichkeit, erste Untersuchungen und Vorverarbeitungen von Datensätzen über eine Anwendung mit einer grafischen Benutzeroberfläche (GUI) auf einem Windows-10/11-Betriebssystem durchzuführen und so individuellen Programmieraufwand zu reduzieren.

Das Tool richtet sich damit im Schwerpunkt an Benutzer, die aus den Bereichen der Netzwerkanalyse, der IT-Sicherheit und des Machine Learnings (ML) stammen. Es bietet eine Möglichkeit IDS-Datensätze schnell und ohne Programmieraufwände gezielt zu untersuchen. Dabei soll das Tool keine tiefgreifende Exploration von Datensätzen gewährleisten, sondern typische oberflächliche Informationen über einen Datensatz darstellen, ausgewählte Informationen durch geeignete Grafiken visualisieren, ausgewählte Schritte einer Vorverarbeitung übernehmen und den Export zur Weiternutzung eines veränderten Datensatzes ermöglichen. Das Programm soll erweiterbar bleiben, um in Zukunft bei Bedarf weitergehende Funktionen implementieren zu können.

Abb. 1: Kontextdiagramm FlowWorkbench



Quelle: Eigene Darstellung.

2.2 Risikomanagement

Um ein Risikomanagement über den gesamten Projektverlauf zu gewährleisten, wurden potenzielle Risiken identifiziert und in einer Risikoliste erfasst. Die Risiken wurden dabei hinsichtlich möglicher Ursachen und Auswirkungen untersucht. Zusätzlich wurden Eintrittswahrscheinlichkeit und Auswirkungen geschätzt. Die Schätzung wurde auf Basis der Erfahrungen des Projektverantwortlichen durchgeführt. Alle Risiken wurden anhand des Produkts aus Eintrittswahrscheinlichkeit und Auswirkung bewertet. Für die identifizierten Risiken wurden potenzielle Gegenmaßnahmen ermittelt. Weitere Informationen können der Legende der Risikoliste entnommen werden. Das Ergebnis der Risikoanalyse ist in Abbildung 2 dargestellt.

Abb. 2: Ergebnisse der Risikoanalyse

Kategorie	Risiko	mögliche Ursachen	mögliche Auswirkungen	Eintrittswahrscheinlichkeit	Auswirkung	Bewertung E * A	Maßnahmen um Eintrittswahrscheinlichkeit oder Auswirkungen zu reduzieren	Status Monitoring, Verhindert, Eingetreten
technisch	Performanceprobleme bei großen Datensätzen	große CSV-Dateien und speicherintensive Verarbeitung	Anwendung nicht funktional ab bestimmter Datensatzgröße	9	2	18	frühe Tests mit Datensätzen der unterstützten Zielgröße, effiziente Operationen verwenden, maximal freigegebene Größe definieren	M
	Ungeeignete Datensätze	inkompatibles Datensatzformat	Datensatz kann nicht mit der Anwendung verarbeitet werden	7	7	49	Definieren des kompatiblen Formats, Tests mit gängigen Datensätzen	M
	Probleme bei der Integration von Bibliotheken	Inkompatibilitäten zwischen Komponenten	Teile der Anwendung können nicht umgesetzt werden	4	6	24	Komponenten früh prototypisch umsetzen und Kompatibilität prüfen	M
	Fehlerhafte Funktionen	Programmierfehler	falsche, unvollständige Ergebnisse oder Anwendung (teilweise) nicht funktional	5	6	30	Frühzeitiges und kontinuierliches Testen	M
	Probleme beim CSV-Import	abweichende Trennzeichen oder Encoding-Probleme	Datensätze können nicht korrekt geladen werden	6	6	36	verschiedene CSV-Testdateien testen, Validierung der einzulesenden Datensätze, Definition von Vorgaben für Datensätze	M
	Speicherprobleme bei großen Datenmengen	hohe RAM-Auslastung durch Datensatzgröße	Anwendung wird instabil oder beendet sich	5	7	35	Speicherverbrauch überwachen und Datensätze begrenzen	M
	Fehlerhafte Exportfunktionen	fehlerhafte/unvollständige Daten werden exportiert	Export nicht nutzbar, Vertrauensverlust in Anwendung durch invalide Datenausgabe	4	7	28	Exportfunktionen mit Testdatensätzen validieren	M
	Unbeabsichtigte Veränderungen in Datensätzen	Fehler in der Programmierung	Korrupte Export-Daten mit negativen Auswirkungen auf ML-Modelle	4	8	32	Frühzeitiges und kontinuierliches Testen	
	Probleme bei der Windows-Ausführung	Inkompatibilitäten mit Windows, Berechtigungsprobleme	Anwendung ist auf Zielsystem Windows 10/11 nicht lauffähig	5	8	40	frühzeitige Tests unter Windows durchführen	M
organisatorisch	Neue Ideen für zusätzliche Funktionen	Vielfältige Erweiterungsmöglichkeiten	Zeitplan und Projektumfang geraten außer Kontrolle	7	7	49	klare Trennung zwischen Muss- und Zusatzfunktionen	M
	Fehlende oder unvollständige Dokumentation	Zu starker Fokus auf die Implementierung	Relevante Dokumentationen fehlen oder sind mangelhaft	5	8	40	Dokumentation parallel zur Entwicklung pflegen	M
	Änderungen an Anforderungen im Projektverlauf	neue Erkenntnisse oder technische Probleme	bereits entwickelte Komponenten müssen angepasst werden, initiale Anforderungen können nicht erfüllt werden	7	6	42	Änderungen nachvollziehbar dokumentieren, frühzeitige und intensive Auseinandersetzung mit Anforderungen	M
	Unzureichende Versionskontrolle	fehlerhafte GitHub-Nutzung oder fehlende Commits	Verlust von Entwicklungsständen, Rückschritte in der Entwicklung	5	8	40	regelmäßige Commits und Backups durchführen	M
	Unrealistische Aufwandsschätzung	fehlende Erfahrung in der Softwareentwicklung, Schätzung durch Einzelperson	Verzögerungen bei Arbeitspaketen und ggfs. im gesamten Projekt	6	7	42	Zeitpuffer einplanen und Prioritäten setzen	M
personelle	Krankheitsbedingter Ausfall	Projekt gänzlich von Einzelperson abhängig	Verzögerungen bei Arbeitspaketen und ggfs. im gesamten Projekt	2	2	4	Zeitreserven einplanen	M
	Einarbeitungsaufwand in neue Technologien	fehlende Erfahrung mit notwendigen Technologien	längere Entwicklungszeiten, nicht Erfüllung von Anforderungen	8	8	64	frühzeitig Prototypen und Tutorials nutzen, Anwendungsumfang realistisch planen	M
	Motivationsverlust bei längerer Entwicklungsphase	hoher Dokumentations- und Implementierungsaufwand	geringere Produktivität	2	3	6	Zwischenziele definieren und Fortschritte dokumentieren	M
rechtlich	Lizenzprobleme durch Bibliotheken	ungeprüfte Nutzung externer Pakete	Verletzung von Lizenzbedingungen	2	7	14	nur etablierte Open-Source-Bibliotheken verwenden, Lizenzen immer prüfen	M
	Nutzung ungeeigneter Datensätze	Datensätze enthalten urheberrechtlich problematische Inhalte	Einschränkungen bei Veröffentlichung oder Demonstration	2	6	12	nur frei verfügbare Datensätze verwenden	M
terminlich	Verzögerung einzelner Meilensteine	höherer Aufwand als erwartet	verzögerter Projektabschluss	7	1	7	Strukturierte Abarbeitung der Arbeitspakete	M
qualitativ	Unzureichende Testabdeckung	zu großer Fokus auf Implementierung und zu kleiner Fokus auf Tests	Fehler bleiben unentdeckt	5	7	35	Testframework für zentrale Funktionen einsetzen, regelmäßiges Testen bereits während der Entwicklung	M
Legende:				E = Eintrittswahrscheinlichkeit		A = Auswirkungen auf das Projekt		
> 50 = hoch, = gefährliches Risiko = Risiko verhindern = sofortige Aufmerksamkeit				fast unmöglich		1	vernachlässigbar (unbedeutend)	
31 - 50 = mittel, = beachtliches Risiko, = Risiko reduzieren = regelmäßige Prüfung				unwahrscheinlich		2-3	gering ("kontrollierbar")	
1 - 30 = gering, = akzeptables Risiko, = Risiko akzeptieren = "Risiko im Auge behalten"				wahrscheinlich (50/50)		4-6	mittelmäßig (beträchtliche Abweichung)	
				sehr sicher		7-8	kritisch (gravierende Abweichung)	
				so gut wie sicher		9-10	äußerst schwerer Schaden	

Quelle: Eigene Darstellung.

2.3 Zeitplanung

In Abbildung 3 ist die für das Projekt aufgestellte Zeitplanung dargestellt. Das Projekt wurde in fünf Meilensteine aufgeteilt. Die Schwerpunkte innerhalb der Meilensteine setzen sich aus mehreren Arbeitspaketen zusammen. Der zeitliche Aufwand wurde auf Ebene der Arbeitspakete geschätzt und anschließend auf Ebene der Schwerpunkte summiert. Auf die Einplanung von zeitlichen Puffern wurde verzichtet. Eventuell auftretende Verzögerungen können damit den Projektabschluss verschieben. Dieser Umstand wird jedoch akzeptiert, da kein konkretes Lieferdatum für die Anwendung festgelegt ist.

Abb. 3: Zeitplanung des Projekts

Phase	Zeitraum	KW (2026)	Schwerpunkt	Arbeitspakete	Stunden	KW 22	23	24	25	26	27	28	29	30	31	32	33	34	
1. Konzeptionsphase	25.05.–31.05.	22	Projektdokumentation		15	•													
		22		Projektidee und -übersicht		•													
		22		Risikomanagement		•													
		22		Zeitplanung	•														
	01.06.–07.06.	23	Anforderungsdokument	Stakeholderanalyse	12		•												
		23		Anforderungsanalyse			•												
	08.06.–14.06.	24	Spezifikationsdokument	Datenmodell	22			•	•										
		24		Geschäftsprozesse				•	•										
		24		Geschäftsregeln				•											
	15.06.–21.06.	25		Systemschnittstellen					•										
25			Benutzerschnittstellen					•	•										
22.06.–28.06.	26	Phasenabschluss und Abgabe	Qualitätssicherung der Artefakte	5						•									
	26		1. Einreichung PebblePad						•										
MS1: Konzeption abgeschlossen										◆									
2. Erarbeitungsphase	29.06.–05.07.	27	Architekturdokument	Technologieübersicht	20						•								
		27		Architekturübersicht						•									
		27		Struktur							•								
		27		Verhalten							•								
	MS2: Technische Planung abgeschlossen										◆								
	06.07.–12.07.	28	Implementierung	GUI	50								•	•	•				
		28		Datenimport								•							
	13.07.–19.07.	29		Datenanalyse											•				
		29		Datenvorverarbeitung											•				
	20.07.–26.07.	30		Export													•		
		Phasenabschluss und Abgabe		10												•			
30	Qualitätssicherung des Codes												•						
30	Qualitätssicherung der Artefakte													•					
	30		2. Einreichung PebblePad													•			
MS3: Anwendung implementiert														◆					
3. Finalisierungsphase	27.07.–02.08.	31	Qualitätssicherung	Code optimieren und fertigstellen	30											•	•		
		31		Teststrategie entwickeln											•				
	03.08.–09.08.	32		Testen und protokollieren													•		
		32		Benutzeranleitung													•		
	MS4: Anwendung auslieferungsbereit															◆			
	10.08.–16.08.	33	Abstract	Making-of	8													•	
		33		Kritische Reflexion													•		
	17.08.–23.08.	34	Phasenabschluss und Abgabe	Qualitätssicherung der Artefakte	6														•
		34															•		
	34			3. Einreichung PebblePad															
MS5: Projekt abgeschlossen																	◆		
Gesamt					178														

Quelle: Eigene Darstellung.

Literaturverzeichnis

- Broy, M., & Kuhrmann, M. (2021). Vorgehensmodelle in der Softwareentwicklung. In M. Broy & M. Kuhrmann (Hrsg.), *Einführung in die Softwaretechnik* (S. 83–124). Springer. https://doi.org/10.1007/978-3-662-50263-1_3
- Canadian Institute for Cybersecurity. (2018). *IDS 2017 | Datasets | Research | Canadian Institute for Cybersecurity | UNB*. <https://www.unb.ca/cic/datasets/ids-2017.html>
- Manocchio, L. D., Layeghy, S., Gallagher, M., & Portmann, M. (2025). An empirical evaluation of preprocessing methods for machine learning based network intrusion detection systems. *Engineering Applications of Artificial Intelligence*, 158, 111289. <https://doi.org/10.1016/j.engappai.2025.111289>
- Ring, M., Wunderlich, S., Scheuring, D., Landes, D., & Hotho, A. (2019). A survey of network-based intrusion detection data sets. *Computers & Security*, 86, 147–167. <https://doi.org/10.1016/j.cose.2019.06.005>
- Schrötter, M., Niemann, A., & Schnor, B. (2024). A Comparison of Neural-Network-Based Intrusion Detection against Signature-Based Detection in IoT Networks. *Information*, 15(3), Artikel 3. <https://doi.org/10.3390/info15030164>
- Sommerville, I. (2012). *Software Engineering*. Pearson Deutschland GmbH. <http://ebookcentral.proquest.com/lib/badhonnef/detail.action?docID=5133710>